XYZ to DXF: Conditioning, Bounded Surface Reconstruction and CAD Export of Scattered XYZ Observations

Abstract

XYZ2DXF is a Python desktop and command-line application for transforming irregular Cartesian XYZ observations into a conditioned observation cloud, a regular elevation grid, spatial diagnostic fields and a CAD drawing. Its numerical objective is the reconstruction of a single-valued scalar surface from scattered planar samples. The processing chain combines optional local residual outlier screening, deterministic minimum-distance thinning, automatic survey-footprint construction, interpolation with established SciPy implementations, independent hold-out assessment and streamed ASCII DXF export. Four surface models are available: local radial basis functions (RBFs), piecewise cubic Clough-Tocher interpolation, piecewise linear triangulated irregular network (TIN) interpolation, and horizontal cell aggregation followed by cubic interpolation and Gaussian grid smoothing. Local RBF is the default. Smooth methods are supplemented by a default-enabled safeguard that detects sparse or nearly collinear support, blends toward bounded reference estimates and limits predictions to a local elevation envelope. This safeguard addresses artificial peaks and troughs between sparse observations while explicitly recording corrections. Numba compiles selected numerical loops, and grid products are evaluated and written in batches. This document presents the implemented algorithms, mathematical formulation, parameters, diagnostics, output conventions, installation, operation, executable packaging and verification procedures.

Keywords: scattered data; bathymetry; terrain modelling; radial basis functions; Clough-Tocher interpolation; linear TIN; overshoot control; XYZ; DXF; Tkinter; SciPy; Numba.

1. Scope and mathematical problem

1.1 Observations and reconstructed field

The input is an unordered collection of finite observations:

$$\mathcal{P} = \{(\mathrm{x}_i, z_i)\}_{i=1}^{N}, \qquad \mathrm{x}_i = (x_i, y_i)^{\mathsf{T}}.$$

The program constructs an approximation to a scalar field over the horizontal plane:

$$z = f(x, y), \qquad (x, y) \in \Omega.$$

Typical applications include bathymetric soundings, topographic spot levels and other scalar measurements attached to planar coordinates. The reconstruction can interpolate observations exactly in suitable configurations or approximate them through regularization and aggregation. The final result depends on conditioning, the selected method, the support geometry and the enabled safeguards.

The program performs more than format conversion. It changes sampling density, may reject observations when screening is enabled, constructs an inferred domain, estimates intermediate levels and attaches diagnostics. These operations should be understood separately when the products are used for engineering analysis.

1.2 Coordinate and unit assumptions

Horizontal distances are Euclidean. X and Y must use compatible linear units in a common Cartesian coordinate system. Projected coordinates in metres are the usual choice; the default minimum distance and grid spacing then represent 1 m and 10 m. Longitude and latitude in degrees should be projected before processing. The program does not perform coordinate transformation, geoid conversion, tidal reduction or vertical-datum correction.

Z can represent elevation or depth using the input sign convention. No sign reversal is applied. A negative-down or negative-elevation dataset remains in that convention. All observations must refer to compatible horizontal and vertical references.

The method represents one Z value per XY location. Overhangs, multiple intersecting surfaces and vertical walls with several elevations at exactly the same XY cannot be represented as a single-valued surface without a modelling decision. Coincident XY observations are handled as described in Section 7.

1.3 Intended products

The application produces five files beside the input: retained observations, interpolated grid, confidence diagnostics, DXF drawing and processing report. It does not produce a raster GIS file, contour lines, a watertight solid, a breakline-constrained mesh or a statistical uncertainty model. DXF geometry consists of points, elevation labels and an optional footprint polyline.

2. Software components and processing sequence

2.1 Computational dependencies

The application uses NumPy for arrays and deterministic sorting; SciPy for spatial searches, convex hulls, Delaunay triangulations, interpolation and image operations; Numba for compiled loops; ezdxf for DXF tag serialization; and threadpoolctl for limiting nested BLAS threading. Tkinter supplies the interface. These libraries provide the numerical backends; the script supplies the workflow, conditioning rules, safeguards, diagnostics and file contracts.

The four selectable method identifiers are `rbf`, `cubic`, `linear` and `voxel`. Nearest-neighbour lookup is an internal fallback, not a selectable interpolation method. The available RBF kernels are `thin_plate_spline` and `cubic`; the latter is an RBF kernel and is distinct from the separate Clough-Tocher `cubic` method.

2.2 Ordered workflow

The same processing function serves the GUI and CLI:

1. Validate settings and resolve the input path.
2. Read finite XYZ observations and count ignored records.
3. Optionally screen vertical residual outliers on the original cloud.
4. Apply deterministic horizontal thinning or exact XYZ duplicate removal.
5. Plan the rectangular grid and optionally infer the survey footprint.
6. Write the retained observations to a staged filtered file.
7. Merge coincident XY only for the interpolation controls.
8. Optionally tune RBF regularization and assess a separate holdout.
9. Fit the final surface using all prepared controls.

10. Evaluate grid tiles; apply safeguards and optional Gaussian smoothing.
11. Stream grid coordinates, diagnostics and DXF entities.
12. Finalize the report and publish completed outputs.

This order is consequential. Outlier decisions, when enabled, use the original dense cloud rather than the thinned cloud. The footprint and grid extent use retained observations. Validation excludes withheld elevations from the fitted surface, while the final delivered surface is refitted with all controls.

3. Interface, parameters and defaults

3.1 Window organization

The non-resizable Tkinter window places the input path above three adjacent panels: processing parameters, method selection and interpolation settings. Execution controls and a full-width processing log occupy the lower panel. The tested layout measures 1400 by 662 pixels. Font metrics and platform display scaling can increase the required size; the code uses the larger of its baseline dimensions and the natural widget requirements.

Text uses dark foreground colours on white panels, approximately 11-point body and help text, bold headings and outlined entries. The log requests five text lines. Parameters relevant only to RBF or voxel processing are enabled when that method is selected. Entries support normal copy and paste. Return starts a conversion while idle. Browse selects the input; Run Conversion starts the worker; Cancel requests cooperative cancellation.

3.2 General controls

Parameter	Default and allowed values
Input XYZ file	Existing file path.
Min Dist	1; finite and nonnegative.
Precision	2; integer from 0 to 15.
PDMODE	3; integer from -32768 to 32767.
Grid Spacing	10; finite and strictly positive.
Robust outlier removal	Disabled.
Outer boundary	Enabled.
Interpolation method	Local RBF.
Prevent local overshoot	Enabled.

Min Dist and Grid Spacing use XY coordinate units. Precision controls exported decimal places, not internal numerical accuracy. PDMODE is passed to the drawing header; the broad accepted integer range is a storage constraint rather than a guarantee that every integer denotes a useful CAD display style.

3.3 Method-specific controls

Parameter	Default and allowed values
RBF kernel	`thin_plate_spline`; alternative `cubic`.
RBF neighbours	50; integer from 3 to 512.

Parameter	Default and allowed values
RBF smoothing	0.1; nonnegative finite value or `auto`.
Voxel cell size	`auto`; otherwise finite and positive.
Gaussian sigma	1; from 0 to 4 grid cells.

The actual RBF neighbour count cannot exceed the available controls. Automatic voxel width is half the grid spacing, subject to a small positive floor. Gaussian sigma zero disables the convolution while retaining cell aggregation and the cubic fit.

3.4 Advanced processing settings

The CLI additionally exposes `--batch-size`, default 4096 and minimum 64, and `--validation-samples`, default 256 and minimum zero. Zero validation samples disables predictive holdout assessment. The Python `Settings` object also exposes `max_boundary_cells`, default 2,000,000 and minimum 10,000. This last setting has no GUI or CLI entry.

The GUI accepts decimal commas in floating-point parameter entries by replacing them with decimal points. The CLI expects the normal decimal-point representation. Coordinate-file commas are separators, so decimal commas must not be used inside XYZ values.

4. Input interpretation and numerical conventions

4.1 Accepted records

Each valid row supplies X, Y and Z as its first three fields. Fields may be separated by spaces, tabs or commas. Additional fields are ignored. The reader does not search later columns for numeric data: the first three fields themselves must parse as numbers.

```
# X Y Z in projected metres and the survey vertical datum
500000.00 4500000.00 -5.20
500012.50 4500000.00 -5.35
500000.00,4500012.50,-5.10
500012.50 4500012.50 -5.28 optional_trailing_field
```

Blank rows, rows beginning with `#`, headers and malformed records are counted as ignored. Rows with parsed but non-finite coordinates, including NaN or infinity, are counted separately and omitted. A file without any finite XYZ row causes an error. Text is read as UTF-8 with optional byte-order mark; undecodable characters are replaced, and the affected fields must still pass numeric parsing.

4.2 Precision and formatting

Coordinates, values and numerical fits use float64 arithmetic. The output precision is applied when serializing files:

$$v_{\text{out}} = \text{format}(v, p), \qquad p \in \{0, 1, \ldots, 15\}.$$

Here the format is fixed decimal notation. Output coordinates rounded to a coarse precision can coincide even when their internal coordinates differ. Conversely, requesting 15 decimals does not create physical accuracy beyond the input data or floating-point resolution.

The grid is interpolated from unrounded retained controls, not from the rounded filtered text. DXF observation points are read back from that filtered text to preserve the same exported observation coordinates. Grid XYZ, confidence XYZ and DXF grid points use matching formatted grid coordinates and elevations.

5. Optional robust outlier screening

5.1 Purpose and activation

Screening is disabled by default. When enabled, it detects observations whose vertical deviation is large relative to nearby residual variation after removing a local planar trend. This is useful for isolated spikes in otherwise coherent surveys. It can also remove genuine abrupt features, so the retained cloud and report should be reviewed when this option is used.

The procedure is a single pass on the original observations. All decisions use the same original cloud, rather than repeatedly deleting points and refitting. Sorting by X, Y and Z establishes a stable working order.

5.2 Neighbourhood radius and sample sufficiency

Let A be the original XY bounding-box area. The implemented planar spacing estimate, for at least two points, is:

$$s_A = \sqrt{\frac{\max(A, 10^{-12})}{N}}.$$

For fewer than two points it is 1. The screening radius is:

$$R = \max(5d_{\min}, 3s_A, 0.01).$$

The KD-tree query requests at most 96 nearest entries, including the observation itself. The tested observation is excluded, and only queried neighbours at distance no greater than R are usable. Consequently, there are at most 95 other observations in a local fit. Fewer than six usable neighbours causes the observation to be kept. Clouds with fewer than seven observations bypass screening.

5.3 Weighted plane

For the tested location, let the usable neighbour distances be r and their count be m. Define a local horizontal scale and mean level:

$$a = \sqrt{\max\left(\frac{1}{m}\sum_{j=1}^{m} r_j^2, 10^{-12}\right)}, \qquad \bar{z} = \frac{1}{m}\sum_{j=1}^{m} z_j.$$

Local coordinates and positive weights are:

$$u_j = \frac{x_j - x_i}{a}, \qquad v_j = \frac{y_j - y_i}{a}, \qquad w_j = \frac{1}{1 + (r_j/a)^2}.$$

The plane approximates the centred neighbour elevations using the basis vector:

$$\mathrm{p}_j = (1, u_j, v_j)^{\mathsf{T}}, \qquad z_j - \bar{z} \simeq \mathrm{p}_j^{\mathsf{T}}\mathrm{c}.$$

The plane coefficient vector is $\mathrm{c} = (c_0, c_x, c_y)^{\mathsf{T}}$. The code assembles the weighted normal matrix and right-hand side:

$$G = \sum_j w_j \mathrm{p}_j \mathrm{p}_j^{\mathsf{T}}, \qquad \mathrm{b} = \sum_j w_j \mathrm{p}_j (z_j - \bar{z}).$$

If the smallest eigenvalue of G is no greater than the larger of its largest eigenvalue and 1, multiplied by 10^{-10}, the geometry is treated as deficient and the observation is kept. Otherwise the code adds a 10^{-12} diagonal stabilizer and solves the three-parameter system. This fit is weighted least squares; the robustness comes from the residual decision below, rather than an iterative robust regression.

5.4 Median-centred residual decision

Neighbour residuals, their median and robust scale are:

$$e_j = z_j - \bar{z} - \mathrm{p}_j^{\mathsf{T}}\mathrm{c}, \qquad m_e = \mathrm{median}(e_j).$$

$$s_e = \max\left(1.4826\,\mathrm{median}(|e_j - m_e|), 10^{-8}\right).$$

The tested point is retained when:

$$|z_i - \bar{z} - c_0 - m_e| \le 3.5 s_e.$$

The median absolute deviation provides a robust residual scale; 1.4826 is the usual normal-distribution consistency factor. The radius floor, scale floors and residual threshold are applied in the supplied coordinate units. They are implementation settings, not survey-specific measurement tolerances.

6. Deterministic minimum-distance thinning

6.1 Exact duplicate mode

With Min Dist zero, only exactly identical XYZ triplets are removed. Equality is numerical equality of parsed values, not closeness within a tolerance. Two observations at identical XY with different Z remain in the filtered cloud.

6.2 Positive-distance mode

For positive minimum distance d, assign each observation to a globally anchored square cell:

$$(c_{x,i}, c_{y,i}) = \left(\left\lfloor \frac{x_i}{d} \right\rfloor, \left\lfloor \frac{y_i}{d} \right\rfloor\right).$$

Its ordering score is the squared distance to the centre of that cell:

$$q_i = \left\| \mathrm{x}_i - d\left(\mathrm{c}_i + \tfrac{1}{2}1\right) \right\|^2 .$$

Candidates are sorted lexicographically by cell X, cell Y, score, X, Y and Z. A candidate is retained only if all previously retained observations satisfy:

$$\|\mathrm{x}_i - \mathrm{x}_j\| \geq d.$$

The rejection test is strictly less than d, so a separation equal to d is allowed. Only the current cell and its eight adjacent cells need be searched. A Numba typed dictionary and linked lists retain cell membership across processing batches.

6.3 Consequences

Thinning is horizontal: a large vertical difference cannot rescue two points closer than d in XY. The retained set is a deterministic greedy subset, not a random sample, an average or a globally optimal packing. Reordering input rows does not change the result for an equivalent numerical cloud. Translating coordinates relative to the globally anchored cell lattice can change the greedy order and therefore the selected subset.

The serial acceptance sequence is deliberate. Parallel acceptance could retain incompatible candidates or change which points survive. Increasing Min Dist reduces redundant support and computation, but can remove narrow features and alter the subsequent footprint, validation and surface.

7. Interpolation control preparation

After filtering, the program sorts retained observations by XY and groups exactly coincident horizontal locations. Each such group contributes one interpolation control with median Z:

$$z_{\mathrm{control}}(x, y) = \mathrm{median}\{z_i : (x_i, y_i) = (x, y)\}.$$

This ensures a single-valued control field and avoids duplicate-coordinate singularities. It does not rewrite the retained-observation product. The report records the number of observations combined for interpolation. Distinct but nearly coincident XY are not merged automatically.

Voxel aggregation, when selected, is applied after this median-XY step. Thus voxel means weight the distinct XY controls equally, rather than giving extra weight to the number of coincident records that originally occupied one XY location.

8. Survey-footprint reconstruction

8.1 Occupancy raster

When enabled, the boundary approximates an enclosing survey footprint using the retained observations. Its initial cell width is the positive Min Dist; if Min Dist is zero, the planar spacing estimate is used. A minimum cell width of 10^{-6} is enforced.

The raster has a 35-cell margin and dimensions:

$$n_x = \left\lceil \frac{x_{\max} - x_{\min}}{c} \right\rceil + 72, \qquad n_y = \left\lceil \frac{y_{\max} - y_{\min}}{c} \right\rceil + 72.$$

Before allocation, c is enlarged until the raster contains no more than `max_boundary_cells`. Observations mark occupied cells. This makes boundary resolution adaptive to survey extent and the memory limit.

8.2 Closing and hole filling

The raw occupancy is first hole-filled. If multiple connected components remain, square morphological closing is tried with radii from 1 to 32 cells. Closing consists of dilation followed by erosion; the original occupied cells are unioned back into the result, and enclosed holes are filled again. SciPy maximum and minimum filters implement the square closing, while connected-component labelling uses the default two-dimensional connectivity. The general closing operation can be written:

$$M_r = \mathrm{Fill}\left[(M_0 \oplus S_r) \ominus S_r \ \cup \ M_0\right].$$

The added raster margin prevents the closing operation from eroding away the outer observed cells. If necessary, the cell size is increased by a factor of 1.35 and the process retried, for up to six attempts. Each attempt reports resolution, dimensions, component count and closing radius.

8.3 Loop extraction and fallback

Exposed cell edges are traced into closed loops. Collinear intermediate vertices are removed, and the loop with the largest enclosed area is retained. A candidate footprint is accepted only after checking that it contains every retained observation. Points on closed raster-cell edges are included using a small cell-coordinate tolerance.

If a suitable connected raster footprint cannot be constructed, a reported convex-envelope fallback is attempted. Rank-deficient data cannot provide a two-dimensional convex boundary and may require disabling boundary use.

8.4 Meaning of the domain

The inferred footprint can fill holes, bridge gaps and connect separate survey groups. It does not distinguish land from water, islands from missing measurements, or unsurveyed obstacles from valid interpolation regions. It is not an imported coastline or breakline constraint. It clips grid products only; it does not remove retained observation points.

The footprint must also be distinguished from the interpolation-control convex hull. A grid node can lie inside the raster footprint and outside the control hull. Such a node is exported when permitted by the footprint, but receives exterior-support diagnostics and may use a fallback.

9. Regular-grid construction

Let h be Grid Spacing and let the extrema refer to retained observations. The nominal lower and upper limits include a margin of 1.5h:

$$x_0 = x_{\min} - 1.5h, \qquad x_u = x_{\max} + 1.5h.$$

The same construction is applied in Y. The implemented dimensions are:

$$N_x = \left\lceil \frac{\max(x_u - x_0, 1)}{h} \right\rceil + 1, \qquad N_y = \left\lceil \frac{\max(y_u - y_0, 1)}{h} \right\rceil + 1.$$

The minimum width of 1 is in coordinate units. Nodes are:

$$(x_i, y_j) = (x_0 + ih, y_0 + jh), \qquad 0 \le i < N_x, \quad 0 \le j < N_y.$$

The ceiling rule can place the last node beyond the nominal upper limit by less than one additional spacing. The lattice is anchored to the observed minima and is not snapped to round global coordinate multiples.

Output order is increasing X, with increasing Y inside each X column. Boundary-enabled runs discard nodes outside the inferred footprint. Boundary-disabled runs keep the complete planned rectangle. No file header is added to the plain grid XYZ product.

Extents and node counts are checked for finite arithmetic, 64-bit count limits and spacing below coordinate floating-point resolution. These checks prevent invalid grids but do not guarantee that an extremely large grid can be processed within practical disk or time limits. If no lattice node falls inside the footprint, processing stops with an instruction to reduce grid spacing or disable clipping.

10. Coordinate normalization and geometric support

10.1 Isotropic scaling

For each fitted surface, the XY origin is the midpoint of its control bounding box. One common length scale s is estimated from the median positive nearest-other-control distance. At most 4096 deterministically spaced controls are sampled for that estimate, with a floor of 10^{-9}. For a single control, s is 1.

$$\mathrm{u}_i = \frac{\mathrm{x}_i - \mathrm{o}}{s}, \qquad v_i = z_i - z_0, \qquad z_0 = \mathrm{median}(z_i).$$

The same scale is used in both horizontal directions. This preserves angles, relative distances and isotropic geometry while reducing large projected-coordinate offsets. Z is centred but not rescaled. Surface results are converted back to the original Z reference after evaluation.

For voxel mode, normalization is estimated from the aggregated controls. For tuning and validation, each training surface estimates its normalization from its own training controls. The numeric smoothing parameter therefore operates on the normalized kernel system of that fit.

10.2 Rank and convex support

Singular values of the centred control coordinates determine spatial rank. A relative threshold of 10^{-12} is used. Rank two supports a planar hull; rank one is represented by endpoints along the principal direction; rank zero is a single location.

A two-dimensional convex hull provides inside/outside classification and distance to its boundary. This does not require a full global Delaunay triangulation for RBF. TIN methods do construct that triangulation. The outside-distance calculation projects a query onto each hull edge segment and uses the minimum distance. Distances are returned to physical XY units when forming the exterior penalty.

11. Local radial basis function interpolation

11.1 Local approximation space

For query location u, SciPy selects the k nearest controls, with k equal to the requested neighbour count or the number of controls if smaller. Over that neighbourhood, the approximation is:

$$f(\mathrm{u}) = z_0 + \sum_{j=1}^{k} a_j \phi(\|\mathrm{u} - \mathrm{u}_j\|) + b_0 + b_x u_x + b_y u_y.$$

The radial terms model curvature, and the affine polynomial supplies the linear trend. The implementation uses `RBFInterpolator` with polynomial degree 1 for both kernels [1,2]. Every unique XY control remains eligible for local selection; the program does not impose a global truncation of the control cloud.

11.2 Kernels

The default thin-plate-spline kernel is:

$$\phi_{\mathrm{TPS}}(r) = r^2 \ln r \quad (r > 0), \qquad \phi_{\mathrm{TPS}}(0) = 0.$$

The alternative cubic RBF kernel is:

$$\phi_{\mathrm{RBF3}}(r) = r^3.$$

Both operate on dimensionless normalized horizontal distances. No user-adjustable shape parameter is exposed. A thin-plate radial kernel should not be confused with the separate Clough-Tocher cubic construction or with Gaussian grid smoothing.

11.3 Augmented linear system

For a local set, define the radial interaction matrix and affine basis matrix:

$$K_{ij} = \phi(\|\mathrm{u}_i - \mathrm{u}_j\|), \qquad P_{i,:} = (1, u_{x,i}, u_{y,i}).$$

The coefficients satisfy:

$$\begin{bmatrix} K + \lambda I & P \\ P^{\mathsf{T}} & 0 \end{bmatrix} \begin{bmatrix} \mathrm{a} \\ \mathrm{b} \end{bmatrix} = \begin{bmatrix} \mathrm{v} \\ 0 \end{bmatrix}.$$

The side condition removes ambiguity between the radial and polynomial components:

$$P^{\mathsf{T}}\mathrm{a} = 0.$$

The polynomial block requires non-collinear planar support. Globally rank-two data can still contain a locally collinear nearest-neighbour set; the retry strategy in Section 16 handles that case. A nonzero smoothing parameter does not remove the polynomial rank requirement.

11.4 Regularization and exactness

With smoothing zero, a successful unmodified local RBF fit reproduces the selected control values to numerical precision. With positive smoothing, the system trades exact reproduction for a regularized approximation. The default is 0.1. It is a coefficient in the normalized radial system, not an elevation error tolerance in metres and not a Gaussian standard deviation.

For affine input data, the polynomial can reproduce the trend with zero radial coefficients when support is adequate. However, exterior safeguards can intentionally replace unconstrained affine extrapolation, and final clipping or other processing can change mathematical reproduction properties.

Increasing smoothing often reduces fluctuations, but does not impose a range bound or guarantee that overshoot decreases monotonically at every location. The explicit safeguard in Section 15 addresses the local range separately.

11.5 Local versus global thin-plate behaviour

The thin-plate kernel is associated with the classical bending functional:

$$J(f) = \iint \left(f_{xx}^2 + 2f_{xy}^2 + f_{yy}^2\right)\, dx\, dy.$$

This provides theoretical context for smooth radial approximation [1]. The delivered algorithm is a sequence of local regularized RBF evaluations with query-dependent control sets and additional safeguards. It does not solve one global constrained bending-energy minimization over the whole survey.

Neighbour-set changes can create joins between local fits. Consequently, smoothness of the kernel does not establish global first-derivative continuity of the delivered local surface. Blending and range limits can introduce additional changes in slope. RBF is therefore described here as a locally smooth reconstruction rather than a globally smooth spline with guaranteed continuity across every neighbourhood transition.

11.6 Practical interpretation

Larger neighbourhoods can stabilize a fit and span survey lines more effectively, but cost more and can mix terrain regimes across real discontinuities. Smaller neighbourhoods preserve locality but are more vulnerable to rank deficiency or uneven support. The default count of 50 is a general starting point, not an optimum for every survey. Automatic smoothing evaluates a small candidate set using held-out data as explained in Section 17.

12. Clough-Tocher cubic interpolation

12.1 Triangulation and cubic patches

The `cubic` method constructs a Delaunay triangulation and evaluates SciPy's `CloughTocher2DInterpolator`. The general construction divides each triangle into subtriangles and forms compatible cubic Bernstein-Bezier patches. A cubic patch can be expressed using barycentric coordinates:

$$p(\alpha, \beta, \gamma) = \sum_{i+j+k=3} b_{ijk} \frac{3!}{i!j!k!} \alpha^i \beta^j \gamma^k.$$

Here the barycentric coordinates sum to one and are nonnegative inside the relevant subtriangle. The coefficients encode values and derivative information. The gradient-estimation principles are described in the interpolation literature [4,5]. The script delegates construction and coefficient calculation to SciPy rather than implementing its own cubic solver.

12.2 Continuity and gradient estimation

The unmodified interpolator is continuously differentiable over its triangulated domain. SciPy estimates nodal gradients through a global procedure intended to approximately minimize curvature [3]. The script sets gradient-estimation tolerance to 10^{-6} and the maximum iteration count to 400. These are solver controls, not acceptable survey errors.

Although evaluation is piecewise, the estimated gradients depend on the surrounding triangulation. A diagnostic neighbour count of three therefore denotes the selected triangle representation; it is not a claim that only three observations influenced the cubic gradients.

12.3 Overshoot and protection

Cubic interpolation can exceed the elevations at its triangle vertices. This can be visually smooth yet physically implausible between sparse survey lines. With protection enabled, the containing triangle supplies a vertex range and a linear reference. Poor support progressively favours that reference, and an out-of-range cubic value fully selects the bounded estimate.

The protected result is not guaranteed to retain the original cubic interpolator's continuous first derivatives. Clipping and adaptive blending are explicit tradeoffs between smoothness and boundedness.

12.4 Exterior and failure handling

SciPy returns non-finite fill values outside the triangulated hull. The application substitutes nearest observations and flags those predictions. A cubic-construction warning causes a reported linear-TIN fallback. A non-finite individual cubic evaluation is also tried through the linear model before nearest fallback. Full Delaunay storage makes this method more demanding than local RBF for very large control sets.

13. Linear TIN interpolation

13.1 Barycentric formulation

The `linear` method uses `LinearNDInterpolator` over a Delaunay triangulation [6]. Let a triangle have vertices u1, u2 and u3. The query barycentric coordinates satisfy:

$$\mathrm{u} = \alpha \mathrm{u}_1 + \beta \mathrm{u}_2 + \gamma \mathrm{u}_3, \qquad \alpha + \beta + \gamma = 1.$$

The interpolated elevation is:

$$z_{\mathrm{TIN}} = \alpha z_1 + \beta z_2 + \gamma z_3.$$

Inside the triangle, the weights are nonnegative. Therefore:

$$\min(z_1, z_2, z_3) \le z_{\mathrm{TIN}} \le \max(z_1, z_2, z_3).$$

This convex-combination property gives an intrinsic range bound within each triangle. The surface is continuous across shared edges, while its gradient generally changes between triangles. It reproduces affine planes inside supported triangles and individual vertex elevations to numerical precision.

13.2 Appropriate use

Linear TIN is a useful reference for sparse bathymetric or topographic datasets, especially when artificial smooth extrema are unacceptable. It can appear faceted, and long skinny triangles can bridge large gaps. Boundedness alone does not establish that the interpolated levels represent the true unsurveyed terrain.

The extra smooth-method guard is skipped for this method because the interior interpolation is already bounded by its vertices. Outside the hull, nearest fallback is flagged. Disabling the general overshoot option does not convert linear TIN into an unbounded exterior extrapolator.

14. Cell aggregation, cubic interpolation and Gaussian smoothing

14.1 Horizontal cell aggregation

The `voxel` method reduces dense control clouds before constructing a cubic surface. In this implementation, voxel means a square cell in XY; the grouping is not a three-dimensional volumetric voxelization. With cell width v:

$$(i_x, i_y) = \left(\left\lfloor \frac{x}{v} \right\rfloor, \left\lfloor \frac{y}{v} \right\rfloor \right).$$

Each occupied cell contributes the arithmetic mean of its actual controls:

$$(\bar{x}_c, \bar{y}_c, \bar{z}_c) = \frac{1}{n_c} \sum_{i \in c} (x_i, y_i, z_i).$$

The mean location is not displaced to the geometric cell centre. For exactly affine data, averaging X, Y and Z in this way preserves the affine relationship. The algorithm subtracts a common origin before summation to reduce the numerical effect of large coordinate offsets.

Only occupied groups are stored. No dense aggregation matrix covering the complete survey rectangle is allocated. The cell lattice is anchored at coordinate zero. Automatic width is:

$$v = \max(h/2, 10^{-9}).$$

Aggregation changes the surface controls but does not change the retained-observation output. Larger cells reduce computation and detail. Arithmetic means can still be affected by erroneous levels; aggregation is not robust outlier rejection.

14.2 Cubic fit on aggregate controls

A Clough-Tocher surface is fitted to the means. Its triangulation, normalization scale, convex support and support spacings refer to those means. The original observation footprint remains the output-clipping domain. These two domains can differ, particularly near the perimeter.

The selected method needs a full triangulation of the reduced controls. Aggregation can make that tractable, but an extremely fine cell width can leave almost the original number of controls.

14.3 Finite Gaussian convolution

For positive sigma, the protected cubic grid is smoothed using a finite, separable Gaussian kernel [7]. Sigma is measured in grid cells; its nominal physical length is sigma multiplied by h. The stencil radius is:

$$R_G = \left\lfloor 4\sigma + \tfrac{1}{2} \right\rfloor.$$

Unnormalized one-dimensional weights and their two-dimensional products are:

$$g_p = \exp\left(-\frac{p^2}{2\sigma^2}\right), \qquad W_{pq} = g_p g_q, \qquad -R_G \le p, q \le R_G.$$

Let M indicate valid cells inside the planned grid and, when enabled, inside the footprint. The normalized masked result is:

$$z_s(i,j) = \frac{\sum_{p,q} W_{pq} M_{i+p,j+q} z_{i+p,j+q}}{\sum_{p,q} W_{pq} M_{i+p,j+q}}.$$

Equivalently, it is convolution of masked elevations divided by convolution of the mask. Exterior cells have constant zero padding in both operations. Normalization prevents those zeros from pulling boundary levels toward zero and preserves a constant field wherever the denominator is positive. Sigma zero bypasses the convolution.

14.4 Tiles, validation and flags

Each exported tile includes a halo of the full stencil radius. The halo is evaluated in bounded sub-batches, convolved, and cropped back to the output tile. This avoids tile-edge artefacts from truncating the kernel at arbitrary batch boundaries. Equivalent on-grid coordinates are snapped to the canonical lattice in holdout-stencil evaluation to avoid last-bit differences.

Validation applies the same finite Gaussian operator around each withheld XY location, using offsets in physical grid spacing and the same domain mask. A withheld elevation is not included

in training cell means. For an off-grid validation location the stencil is translated to that location; for an actual grid node it agrees with the grid operator to numerical precision.

Fallback and correction flags propagate when any included stencil sample carries the corresponding flag. The guard is applied before and after Gaussian convolution. This last step prevents averaging across different neighbourhood envelopes from bypassing the envelope defined at the destination location.

14.5 Properties

Normalized Gaussian smoothing is a positive weighted average of valid stencil values. It attenuates small-scale fluctuations, but can also blur genuine features, shift slopes and alter extrema. The combined voxel method is an approximation; it is not expected to reproduce every original observation. The subsequent local range correction can further alter a smoothed value.

15. Local overshoot and sparse-support protection

15.1 Motivation and design

A smooth interpolator does not generally obey a maximum principle. Sparse cross-sections, parallel contour bands, strongly uneven densities and nearly collinear local controls can permit artificial peaks or troughs. Positive RBF smoothing alone is not a bound.

The default-enabled protection supplements `rbf`, `cubic` and `voxel` with an application-specific local guard. It is a numerical policy implemented by this program, not an intrinsic guarantee supplied by the library interpolators. Its inputs are local ranges, distance relative to sampling density, support anisotropy and a bounded reference estimate.

15.2 Initial support and local ranges

The initial support size is:

$$k_g = \min\left(n, \max\left(16, \min(k_{\text{settings}}, 64)\right)\right).$$

For default RBF settings this is 50 controls. The GUI uses the default neighbour setting for the other methods. The first local envelope is the minimum and maximum of those neighbours' centred elevations. Each control also has a nearest-other-control spacing, computed on the full fitted support.

Let d1 be the distance from the query to its nearest control, and let sloc be the median control spacing in the guard neighbourhood. The dimensionless distance ratio is:

$$\rho = \frac{d_1}{\max(s_{\text{loc}}, 10^{-12})}.$$

The distance-risk ramp is:

$$t_d = \min\left(1, \max\left(0, \frac{\rho - 1.5}{2.5}\right)\right).$$

It begins at a ratio of 1.5 and reaches full weight at 4. These are program thresholds, not estimated physical uncertainty limits.

15.3 Geometric elongation

For guard-neighbour positions, form the unnormalized two-dimensional covariance sums xx, yy and xy about their mean. Their eigenvalue ratio can be written:

$$a_s = \frac{\max(T - D, 0)}{\max(T + D, 10^{-24})}, \qquad T = xx + yy.$$

$$D = \sqrt{\max((xx - yy)^2 + 4xy^2, 0)}.$$

The factor common to both covariance eigenvalues cancels in this ratio. Small values indicate elongated or nearly collinear support. The elongation ramp and smooth transition weight are:

$$t_a = \min\left(1, \max\left(0, \frac{0.03 - a_s}{0.03}\right)\right).$$

$$t = \max(t_d, t_a), \qquad w = t^2(3 - 2t).$$

At a normalized nearest distance no greater than 10^{-10}, the reference is the nearest control and the risk weight is reset to zero. This prevents the sparse-distance criterion alone from changing an observation location. An out-of-range prediction can still trigger the range rule.

15.4 Positive-weight reference

Before a triangle reference is available, the guard forms a positive inverse-square-distance average. The code uses relative weights to avoid unnecessarily large magnitudes:

$$\omega_j = \left(\frac{\max(d_1, 10^{-12})}{\max(d_j, 10^{-12})}\right)^2.$$

$$v_{\mathrm{IDW}} = \frac{\sum_j \omega_j v_j}{\sum_j \omega_j}.$$

These weights are nonnegative, so the estimate lies within the local range. A query effectively coincident with a control uses that control directly. This IDW calculation is a safeguard reference, not an additional selectable interpolation method.

15.5 Triangle references

For cubic and voxel-cubic methods, the containing global TIN triangle supplies the safe linear value and the minimum and maximum of its three vertex elevations. The risk still reflects local sampling support. Exterior TIN queries already use nearest fallback; they do not receive a new sparse blend toward an invented exterior triangle.

For RBF, local linear reference models are built only for risky or initially out-of-range queries. Up to 512 nearest controls are queried. If their small local triangulation contains the query, its linear value and vertex range replace the initial reference and range. At most 32 such models are retained in a least-recently-used cache, keyed by the selected controls. This avoids a full global Delaunay triangulation in the RBF path.

If the expanded interior support does not enclose the query, the positive-weight reference remains available. Risky exterior RBF queries use the nearest observation as their reference with full weight. Exterior RBF predictions that do not trigger risk or range checks may remain extrapolated; the guard is not a universal ban on RBF extrapolation.

15.6 Final correction

Let vraw be the centred smooth value and let L and U be the selected local bounds. For range comparisons the code uses:

$$\epsilon_z = 10^{-10} \max(1, |L|, |U|).$$

If vraw lies outside the bounds beyond this tolerance, w is set to one. The corrected prediction is:

$$v_c = \min\left(U, \max\left(L, (1-w)v_{\text{raw}} + wv_{\text{safe}}\right)\right).$$

Finally the elevation offset is restored. A change exceeding the comparison tolerance sets `clampedLocal=1` and `fallback=1`. Complete replacements update neighbour and patch counts to the selected reference representation. Tiny range clips below the tolerance need not set a correction flag.

The guard may replace the initial nearest-neighbour range by the range of an enclosing triangle. Thus its formal bound refers to the selected local support, not to the initially queried set under all circumstances. It also does not imply monotonic variation along every possible profile or bound the surface gradient.

15.7 Determinism and continuity

At shared triangle edges, more than one triangle can contain a point within numerical tolerance. The guard traverses adjacent eligible triangles and chooses a canonical triangle index, preventing batch order from selecting different vertex envelopes. This improves consistency among tiles and validation stencils.

The smoothstep weight has smooth endpoints, but neighbourhood changes, hull transitions and explicit clipping can still change derivatives or introduce local joins. Protection prioritizes plausible local ranges; it cannot infer unknown seabed structures between soundings or repair bad control elevations. Disable it through the GUI checkbox or `--allow-overshoot` only when the unconstrained interpolation behaviour is intended.

16. Numerical fallbacks and exceptional geometries

16.1 Rank-deficient support

One-point, coincident-location or collinear datasets do not support a two-dimensional degree-one RBF or ordinary planar triangulation. The surface uses nearest-observation values and flags fallback predictions. Boundary construction may separately reject these data unless disabled. Such a successful surface fallback does not imply that the footprint can be constructed.

16.2 Singular local RBF neighbourhoods

RBF evaluation is batched. If a batch raises a local linear-algebra or value error, the code recursively splits the query set so that one problematic location does not invalidate unrelated queries. A failed single query is retried with a larger support, approximately doubling the count, up to the smaller of 512 and the number of controls. A successful expanded fit is flagged. If all attempts fail, nearest fallback is retained.

This strategy addresses local polynomial rank deficiency and some ill-conditioned neighbourhoods. It is not a proof that every extreme coordinate distribution is numerically well-conditioned.

16.3 TIN and cubic failures

Failed Delaunay construction produces nearest fallback. Warnings during cubic gradient estimation select the linear model and flag the substitution. Individual non-finite cubic values are tried with linear interpolation, then nearest values if necessary. Ordinary exterior TIN fill values use nearest fallback.

Nearest-distance ties in fallback selection use a small relative tolerance and choose the lowest stable control index. The script checks final predictions for finite Z values before export.

16.4 Scope of fallback flags

A fallback flag can identify an expanded neighbourhood, substituted interpolator, sparse-support correction or inherited Gaussian-stencil condition. It does not exclusively mean nearest-neighbour interpolation. Conversely, the numeric correction flag records changes above the guard tolerance, not every theoretical risk score above zero.

17. Parameter tuning and independent holdout assessment

17.1 Holdout construction

Let n be the number of unique XY controls before any voxel reduction. The requested holdout count is:

$$n_v = \min(n_{\text{requested}}, \lfloor n/10 \rfloor).$$

A random permutation with fixed generator seed 1729 establishes the split. Stable control ordering makes repeated equivalent runs reproducible in the same numerical environment. A holdout smaller than two, or fewer than twelve controls overall, bypasses predictive assessment. In practice the ten-percent rule requires at least twenty controls to obtain two withheld points.

The test controls are excluded from the fitted validation surface. For voxel mode, only training controls contribute to aggregation. Preprocessing decisions, the footprint and the grid have already been constructed from the conditioned dataset; this is not a complete blind validation of the entire data-acquisition and conditioning chain.

17.2 Automatic RBF smoothing

When smoothing is `auto`, a separate tuning set of the same size is taken after the final test set in the permutation. Each candidate is fitted without either held-out set. The candidate values are:

$$\lambda \in \{0, 0.01, 0.1, 1\}.$$

For tuning errors e, the selection score is:

$$J = \mathrm{RMSE} + 0.35 P_{95}(|e|) + 0.15\mathrm{MAE}.$$

A score tie within 10^{-10} favours 0.1. If assessment is unavailable, `auto` resolves to 0.1. The selection searches only these four values; it is not a continuous optimizer, generalized cross-validation or a local per-query smoothing estimator.

After choosing smoothing, the validation surface is fitted to all controls except the final test set. Tuning controls are then available for this fit. The untouched test elevations provide the reported assessment. Finally, the delivered surface is fitted to all controls.

17.3 Error measures

Signed errors use predicted minus observed elevation:

$$e_j = \hat{z}_j - z_j.$$

The reported aggregate measures are:

$$\mathrm{RMSE} = \sqrt{\frac{1}{n_v}\sum_j e_j^2}, \qquad \mathrm{MAE} = \frac{1}{n_v}\sum_j |e_j|.$$

The 95th percentile of absolute errors is also reported, together with the holdout count and number of fallback predictions. All error metrics use Z units. Predictions include enabled protection and, in voxel mode, the same Gaussian smoothing operator used for export.

17.4 Interpretation

These are predictive errors on conditioned, withheld elevations. They are not residuals from a model fitted to those same elevations. However, a random holdout can be optimistic when dense neighbours remain near each test point. It does not establish accuracy in large gaps or assess independent survey biases, datum errors or navigation errors.

For a rigorous project assessment, retain an external check survey or use a separate spatially blocked validation workflow. Spatial block validation, uncertainty calibration and automatic method selection are not implemented in this application.

18. Spatial residuals and confidence diagnostics

18.1 Local error field

At each exported node, the code queries up to eight nearest validation locations in normalized XY coordinates. With distances dj and holdout errors ej, it uses:

$$w_j = \frac{1}{\max(d_j, 0.05)^2}.$$

The local residual diagnostic is a weighted root-mean-square error:

$$r(\mathrm{x}) = \sqrt{\frac{\sum_j w_j e_j^2}{\sum_j w_j}}.$$

It is not a signed interpolation residual at the exported node, because the true node elevation is normally unknown. The distance floor prevents a single tiny separation from creating unbounded weights. If validation is unavailable, residual is written as `nan`, rather than an artificial zero.

18.2 Exterior-support penalty

Let dout be physical distance outside the final surface controls' convex support, h the grid spacing and s the normalization scale. The penalty is:

$$B = \exp\left(-\frac{d_{\mathrm{out}}}{\max(h, s)}\right).$$

It equals one inside the hull. It is unrelated to distance from the raster footprint edge. In voxel mode, it uses the aggregate-control hull. Rank-deficient support uses distance to its point or line-segment representation.

18.3 Error-quality factor

Let sz denote the population standard deviation of centred surface control values, with a floor of 10^{-8}. The error scale and quality factor are:

$$E = \max(\mathrm{RMSE}, 0.01 s_z, 10^{-8}), \qquad Q = \frac{1}{1 + r/E}.$$

When validation is unavailable, Q is set to 0.25. No measured local error is implied by that value.

18.4 Confidence index

Let dnear be physical distance to the nearest final control. Define F as 0.35 for a fallback prediction and 1 otherwise; define H as 0.25 for rank below two and 1 otherwise. The confidence index is:

$$C = \operatorname{clip}\left[\exp\left(-\frac{d_{\text{near}}}{2s}\right) BQFH, 0, 1\right].$$

The index is a heuristic combination of proximity, hull support, held-out error and algorithmic substitution. It is not a probability, confidence interval, standard deviation, calibrated reliability or compliance classification. Its numerical constants are explicit program policies. Comparison among runs should account for changes in normalization, validation errors, aggregation and control density.

18.5 Diagnostic columns

The confidence file contains twelve columns in this order:

Column	Meaning
`x`, `y`, `z`	Formatted grid coordinates and elevation.
`confidence`	Index C, limited to the interval 0 to 1.
`residual`	Local weighted RMS error in Z units; `nan` if unknown.
`lambda`	Resolved normalized-coordinate RBF smoothing.
`boundaryPenalty`	Exterior convex-support penalty B.
`fallback`	One if a substitution or correction is flagged.
`outsideHull`	One outside the final control convex support.
`clampedLocal`	One for a recorded local correction.
`neighborCount`	Support count of the represented base or replacement fit.
`patchCount`	One for a radial/triangle fit; zero for nearest or IDW.

For non-RBF methods `lambda` is zero. Neighbour counts are the selected RBF count, three for a triangle, one for nearest, or the support count of a positive-weight replacement. They are not necessarily the full number of observations influencing a cubic gradient or Gaussian stencil. Partial blends can retain base counts; full replacements use the reference counts. Gaussian processing propagates flags across its stencil but does not report the union of all contributing controls.

All first seven fields are written at the selected decimal precision; the final five are integers. Low precision can round a small smoothing value or confidence value to zero. The report retains more informative scalar settings and summaries. The confidence header is one physical line; its logical sequence is shown below across two lines for page readability:

```
# x y z confidence residual lambda boundaryPenalty
fallback outsideHull clampedLocal neighborCount patchCount
```

19. Output products and data contracts

19.1 Naming

The output suffix is appended to the full input path, including its existing extension. For `survey.xyz`, the outputs are:

```
survey.xyz.filtered.xyz
survey.xyz.grid.xyz
survey.xyz.confidence.xyz
survey.xyz.dxf
survey.xyz.rpt.txt
```

The input is never replaced by these products. A successful rerun replaces existing products with the same names. The destination directory is the input directory, not the directory from which Python was launched. It must allow temporary files and output replacement.

19.2 Retained observations

`filtered.xyz` contains finite observations after optional screening and thinning. It has three space-separated columns and no header. Z values are retained observation values, not smoothed predictions. With zero Min Dist, distinct XYZ at one XY remain here even though their interpolation representation uses median Z.

19.3 Grid

`grid.xyz` contains the interpolated XYZ nodes, in X-major/Y-minor order and optionally clipped to the footprint. Missing nodes outside the footprint are omitted entirely rather than written as NaN. The selected method and safeguards determine Z; precision determines only formatting.

19.4 Confidence table

`confidence.xyz` contains one header and one record per exported grid node in the same order. Its first three formatted values match `grid.xyz`. It is suitable for joining by row order or parsed coordinates when post-processing diagnostics, but duplicate rounded coordinates can occur at low precision.

19.5 Drawing

The DXF contains retained observation POINT/TEXT pairs, interpolated POINT/TEXT pairs and the optional boundary. It does not contain every raw input point if filtering removed observations. Its geometry conventions are detailed in Section 20.

19.6 Processing report

`rpt.txt` is UTF-8 text with settings, dependency versions, input counts, conditioning counts, grid plan, footprint iterations or fallback, control normalization, tuning and holdout metrics, diagnostic summaries, entity counts and duration. The mean residual summary is the arithmetic mean of the nodewise RMS diagnostics; it is not a new independently estimated global RMSE.

A successful report and the final GUI/CLI log include:

```
Finished in 0.12 minutes.
```

The value is calculated from monotonic elapsed seconds divided by 60 and formatted with two decimals. Reported computational elapsed time is measured before the final output-publication step. It therefore does not include all possible time spent in final filesystem replacement. Phase progress is an estimate of workflow position, not a remaining-time forecast.

20. DXF construction and CAD conventions

20.1 Format and streaming

The drawing is ASCII DXF R12, identified by `AC1009`. The script uses ezdxf's tag serializer and writes bounded batches of entity records [11]. The complete drawing is never accumulated as a Python document object. Explicit fixed-decimal serialization preserves the requested coordinate precision.

20.2 Layers

Layer	Entity and colour index
`xyz_points`	Retained-observation POINT; 7.
`xyz_labels`	Observation elevation TEXT; 3.
`boundary`	Closed footprint POLYLINE; 1.
`grid_points`	Interpolated POINT; 5.
`grid_labels`	Grid elevation TEXT; 4.

Colour indices follow the CAD application's display palette; index 7 commonly adapts to the background. The script also defines the default layer, continuous linetype and standard text style.

20.3 Points, labels and boundary

POINT entities retain their three-dimensional XYZ. Labels display the corresponding formatted Z value and are positioned at:

$$\mathrm{x}_{\mathrm{label}} = (x + 0.2, y + 0.2, 0), \qquad H_{\mathrm{text}} = 1.0.$$

For source labels, offsets are applied to coordinates read from the rounded filtered file; for grid labels they are applied to the evaluated grid coordinates before formatting. Label positions and heights use drawing coordinate units and are not automatically scaled with grid spacing.

The optional boundary is one closed two-dimensional POLYLINE at Z zero. Its elevation is a drawing convention, not an interpolated contour. `$PDMODE` uses the input integer, while `$PDSIZE` is fixed at 0.5. These names are CAD header variables, not numerical formulas.

20.4 Viewport and interpretation

An active viewport is centred on the enclosing extent of observation points, planned grid and boundary. Its nominal view size is 1.1 times the larger horizontal extent, with a minimum of 1. The planned grid can exceed the clipped exported grid, so the initial CAD view may contain a surrounding margin.

Point and text entities can create a large drawing for dense grids. Labels may overlap even though their numeric values are correct. Turning off label layers in CAD improves visibility. No triangulation, raster, surface shading or contour generation is embedded in the DXF. The script does not assign a geospatial CRS or convert drawing units.

21. Execution architecture and output publication

21.1 GUI worker model

The Tk main thread owns all widgets. A background thread runs the shared processing function and places progress, completion and error events in a thread-safe queue. A scheduled Tk poll consumes the queue and updates the log, status, controls and progress bar. Worker code does not directly operate Tk widgets.

Input controls are disabled during processing. Cancellation uses a threading event checked between file-reading intervals, numerical batches, boundary iterations and other stages. Native SciPy calls and already-running compiled batches finish before the next check. Cancellation is responsive at those boundaries rather than instantaneous at every instruction.

Closing during a run requests confirmation, signals cancellation and waits for worker cleanup before destroying the window. Dependency import failures provide installation guidance; an unavailable display can be bypassed with CLI processing.

21.2 Staged writes

Temporary products are written into a temporary directory beside the source. A normal successful run closes all products before replacing destination files. Existing products are moved to temporary backups during publication. If an operating-system replacement fails, the code attempts to restore the previous files in reverse order.

Each file rename uses same-filesystem replacement. The complete five-file publication is a coordinated sequence with rollback, not one filesystem transaction simultaneously visible to all readers. A process kill, system crash or disk failure during rollback is outside the guarantees of normal exception handling.

21.3 Failures and reports

Ordinary processing errors and cooperative cancellation discard incomplete new data files and preserve prior data products. When failure occurs while the staged report is still open, that report records the error or cancellation and is published as the current report. A failure during final publication occurs after the report was closed; the publication rollback can instead restore the previous report together with the previous data products.

The CLI returns zero on success, one for handled processing failures, and 130 for interruption or cancellation. Argument-parser errors normally return two. Command-line interruption is handled separately from the GUI cancellation event and is not guaranteed to leave the same detailed cancellation report.

22. Acceleration, memory and computational cost

22.1 Compiled kernels

Numba `njit` compiles the local plane/residual screening loop, greedy thinning, convex-support classification and distances, guard-neighbour summaries, and canonical triangle selection. These kernels use nopython execution, `nogil=True` and `fastmath=False`. Independent outlier tests additionally use parallel loops. The SciPy interpolators themselves remain SciPy implementations; JIT does not translate the whole application into native code [12,13].

The maximum CPU worker count is the smaller of eight, the available CPU count and the configured Numba thread limit, with a minimum of one. Large outlier KD-tree batches can also use multiple workers. BLAS threading is limited to one within the processing context to reduce nested-thread oversubscription. Deterministic thinning remains serial.

22.2 First-use compilation and caching

The first call for a kernel signature incurs compilation time. Normal Python execution enables the Numba disk cache. Subsequent compatible runs can reuse compiled kernels from a writable cache location. Frozen executables disable this persistent cache in the script, so a new executable process compiles required kernels again. No CUDA device or separately installed C/C++ compiler is required for these Numba CPU kernels; llvmlite supplies the LLVM runtime.

Packaging with PyInstaller does not precompile every numerical kernel or guarantee zero startup delay. Importing scientific libraries and extracting a one-file bundle also contribute to executable startup.

22.3 Memory model

The input cloud, sorting arrays, retained controls, KD-trees and geometric structures remain in memory. Thus the application is not a fully out-of-core point-cloud engine. A raw float64 XYZ array alone requires approximately 24N bytes; temporary arrays and indexes add to this.

Local RBF avoids a dense global radial matrix. A local solve has a matrix of order k plus three polynomial coefficients, rather than N plus three. Dense local factorization has cubic algebraic cost in k; actual repeated-query performance depends on neighbourhood reuse and library implementation. It is not appropriate to describe the entire algorithm as linear time in N.

Cubic and linear methods require global Delaunay storage. Voxel mode reduces the controls first but still triangulates the resulting means. Boundary rasters are bounded by a cell-count limit, although several temporary image arrays are used during morphology.

22.4 Grid scaling

Without clipping, node count grows approximately as horizontal area divided by squared spacing:

$$N_{\mathrm{grid}} \simeq \frac{A_{\mathrm{grid}}}{h^2}.$$

Halving h can therefore produce roughly four times as many nodes. DXF writes two entities per observation or grid node, so file size and CAD loading can dominate for dense outputs. A

smaller batch reduces base query arrays but does not reduce the size of the retained cloud or the global triangulation.

Grid tiles are streamed; a full output-grid array is not allocated. Gaussian tiles include halos and may re-evaluate overlapping samples in adjacent tiles. RBF protection can also add local triangulations in risky areas. Reducing voxel control count or increasing Min Dist may be more effective than reducing batch size when support structures dominate memory.

23. Installation

23.1 Requirements

Use a supported 64-bit Python environment with Tk support. The script requires Python 3.10 or newer; Python 3.12 is the recommended baseline for reproducing the tested environment. Newer Python releases also require compatible wheels for all numerical dependencies. Install Python from python.org with Tcl/Tk enabled, or use a suitable Anaconda environment.

The supplied dependency ranges are:

```
numpy>=1.26,<3
scipy>=1.11,<2
ezdxf>=1.3,<2
threadpoolctl>=3.2,<4
numba>=0.62,<1
```

These are allowed ranges, not an assertion that every possible combination has been tested. Let pip resolve a compatible set, verify it, and retain exact versions for reproducible work. Numba's llvmlite dependency is installed automatically.

23.2 Windows Command Prompt

Place `xyz2dxf_gui.py`, `requirements.txt` and, optionally, the tests in one working directory. Open Command Prompt there:

```
py -m venv .venv
.venv\Scripts\activate.bat
python -m pip install --upgrade pip setuptools wheel
python -m pip install --upgrade -r requirements.txt
python -m pip check
python xyz2dxf_gui.py
```

If the `py` launcher is unavailable, use `python -m venv .venv`. If several Python versions are installed, `py -3.12 -m venv .venv` explicitly selects Python 3.12. A virtual environment isolates project dependencies [15].

To launch later from the same folder without activating the environment:

```
.venv\Scripts\python.exe xyz2dxf_gui.py
```

If only the source file is available, dependencies can be installed directly:

```
python -m pip install "numpy>=1.26,<3" "scipy>=1.11,<2" ^
  "ezdxf>=1.3,<2" "threadpoolctl>=3.2,<4" "numba>=0.62,<1"
```

The caret is the Windows Command Prompt line-continuation character. It must be the last character on the continued line. It is not used in Bash or PowerShell commands.

23.3 Windows PowerShell

A virtual environment can be used without changing the system's execution policy or activating a script:

```
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe xyz2dxf_gui.py
```

23.4 Anaconda Prompt

Use an isolated environment rather than installing numerical dependencies into a shared base environment:

```
conda create -n xyz2dxf python=3.12 tk pip
conda activate xyz2dxf
python -m pip install --upgrade pip setuptools wheel
python -m pip install -r requirements.txt
python -m pip check
python xyz2dxf_gui.py
```

Anaconda Prompt also accepts the Windows virtual-environment procedure. Choose one environment workflow and ensure pip and Python belong to that environment.

23.5 Linux and macOS

A typical shell procedure is:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip setuptools wheel
python -m pip install -r requirements.txt
python xyz2dxf_gui.py
```

Tkinter is supplied by the Python distribution rather than an ordinary `pip install tkinter` package. On Debian/Ubuntu, a missing Tk module generally requires the distribution's `python3-tk` package; virtual-environment support may require `python3-venv`. Other distributions and macOS installations should use a Python build with Tk support [14]. A graphical display is required for GUI launch; the CLI can operate without opening Tk.

23.6 Installation diagnostics

Check the interpreter, imports and Tk independently:

```
python -c "import sys; print(sys.executable)"
python -c "import numpy, scipy, ezdxf, numba, threadpoolctl"
python -m tkinter
python xyz2dxf_gui.py --help
```

The Tk command opens a small test window. If imports fail, install into the interpreter printed by the first command. A clean environment is preferable when incompatible packages are already installed. Save a reproducible environment record after successful verification:

```
python -m pip freeze > environment-lock.txt
```

24. Graphical and command-line operation

24.1 GUI procedure

1. Start the script without arguments.
2. Browse to the XYZ file or paste its path.
3. Confirm coordinate units and choose Min Dist and Grid Spacing.
4. Choose the interpolation method and its active settings.
5. Review outlier screening, boundary clipping and overshoot protection.
6. Select Run Conversion and follow the log.
7. Read the report and inspect filtered observations, grid and diagnostics.
8. Open the DXF and manage point/label layers as needed.

The default workflow uses Min Dist 1, Grid Spacing 10, RBF with 50 neighbours and smoothing 0.1, boundary clipping and overshoot protection enabled, and outlier screening disabled. The GUI does not implement an automatic preview, parameter persistence, interactive map or multi-file selection.

24.2 Basic CLI examples

Run the default conversion:

```
python xyz2dxf_gui.py --input survey.xyz
```

Use automatically selected RBF smoothing:

```
python xyz2dxf_gui.py --input survey.xyz --smoothing auto
```

Choose linear TIN and a finer grid:

```
python xyz2dxf_gui.py --input survey.xyz --method linear --grid-spacing 5
```

Enable screening explicitly:

```
python xyz2dxf_gui.py --input survey.xyz --outliers
```

Retain distinct XYZ observations and the full rectangular grid:

```
python xyz2dxf_gui.py --input survey.xyz --min-dist 0 --no-boundary
```

Use aggregation with the default automatic cell width and Gaussian sigma:

```
python xyz2dxf_gui.py --input survey.xyz --method voxel
```

Evaluate the unconstrained cubic method:

```
python xyz2dxf_gui.py --input survey.xyz --method cubic --allow-overshoot
```

Use quotes around paths containing spaces. For example, in Windows CMD:

```
python xyz2dxf_gui.py -–input "C:\Survey Data\survey.xyz"
```

24.3 Complete option reference

Option	Meaning
`--input`	Input path; enables CLI conversion.
`--min-dist`	Horizontal thinning distance.
`--precision`	Exported decimal places.

Option	Meaning
`--pdmode`	DXF point-display setting.
`--grid-spacing`	Cartesian grid increment.
`--method`	`rbf`, `cubic`, `linear` or `voxel`.
`--outliers`	Enable original-cloud screening.
`--no-outliers`	Explicitly disable screening.
`--no-boundary`	Export the complete rectangular grid.
`--allow-overshoot`	Disable smooth-method protection.
`--neighbors`	RBF local neighbour count.
`--smoothing`	RBF regularization or `auto`.
`--kernel`	`thin_plate_spline` or `cubic` RBF.
`--voxel-size`	Aggregation width or `auto`.
`--gaussian-sigma`	Smoothing width in grid cells.
`--batch-size`	Maximum base evaluation batch.
`--validation-samples`	Maximum final holdout count.
`--help`	Print argparse usage and exit.

Method-specific numerical parameters are validated for the selected method. Nevertheless, CLI text-to-number conversion happens before processing, so malformed numeric text can fail even when that parameter would otherwise be inactive.

Running without any arguments opens the GUI. Supplying processing options without `--input` is an argument error. The help option prints usage without opening the GUI. There is no separate output-directory option, imported-boundary option, random-seed option or nearest-neighbour method option.

25. Building a standalone executable

25.1 Windows one-file GUI build

Build on Windows with the same architecture intended for deployment. From Command Prompt in the source directory, use the following sequence for a fresh environment:

```
py -m venv .venv
.venv\Scripts\activate.bat
python -m pip install --upgrade pip setuptools wheel
python -m pip install --upgrade -r requirements.txt
python -m pip install --upgrade pyinstaller
python -m PyInstaller --noconfirm --clean --onefile ^
  --windowed --collect-all numba --collect-all llvmlite ^
  --name xyz2dxf_gui xyz2dxf_gui.py
```

For an existing verified environment, activate it and run the PyInstaller installation and build steps. Avoid recreating an environment unnecessarily. The expected executable is:

```
dist\xyz2dxf_gui.exe
```

PyInstaller adds the executable extension on Windows. The name argument therefore uses the base name without `.exe`.

25.2 Meaning of the build options

Option	Purpose
`--noconfirm`	Replace build output without an interactive prompt.
`--clean`	Clean PyInstaller caches and temporary build files.
`--onefile`	Produce one distributable executable.
`--windowed`	Suppress the console for the GUI application.
`--collect-all`	Include package modules, data and binaries.
`--name`	Set the executable base name.

The explicit Numba and llvmlite collection requests help include the JIT runtime. Scientific packages and Tk also need their appropriate PyInstaller hooks and runtime files. The resulting bundle can be large. PyInstaller packages an interpreter and dependencies; it is not a whole-program optimizing compiler [16].

25.3 Console build for command-line use

The windowed build is intended for graphical use. The script's CLI prints to standard output and errors, which are unavailable in the normal way in a Windows windowed executable. Build a separate console executable for CLI use or troubleshooting:

```
python -m PyInstaller --noconfirm --clean --onefile ^
  --console --collect-all numba --collect-all llvmlite ^
  --name xyz2dxf_cli xyz2dxf_gui.py
```

Run it from Command Prompt with the normal arguments:

```
dist\xyz2dxf_cli.exe --input "C:\Data\survey.xyz"
```

25.4 Deployment verification

Test the built program on the target Windows environment: start the GUI, browse a real file, run a small conversion with every required method, inspect the five outputs, exercise cancellation and confirm the report. Test the console bundle separately if it will be distributed. Source tests on Linux do not validate a Windows executable.

A one-file bundle extracts runtime components when launched and must run Numba JIT compilation as needed. Target permissions, antivirus policies and temporary-directory access can affect startup. If diagnosing a bundle problem, an `--onedir` console build can make missing libraries easier to identify before returning to one-file packaging. No Windows executable is supplied or claimed as tested by this README.

26. Verification and reproducibility

26.1 Automated tests

If `test_xyz2dxf.py` is present beside the script, run:

```
python -m unittest -v test_xyz2dxf
```

The supplied suite contains 42 tests covering parsing, ignored and non-finite records, horizontal thinning, ordering and batch state, outlier decisions, duplicate XY handling, projected-coordinate plane reproduction, exact zero-smoothing RBF values, singular support, exterior fallbacks and linear-TIN bounds.

It also checks sparse negative-elevation examples, local rather than global bounds, interpolation between contour bands, guard batch-order consistency, Gaussian bounds and tile/stencil agreement, boundary containment and concavity, deterministic holdouts, low confidence when validation is unavailable, all output products, DXF auditing, fifteen-place formatting, elapsed-time messages, CLI execution, cancellation, simulated write failure and publication rollback. Numba tests check native nopython compilation and deterministic outlier screening.

26.2 Recorded execution environment

The numerical/export suite passed in the recorded Linux test environment with Python 3.12.14, NumPy 2.3.5, SciPy 1.17.0, Numba 0.68.1, llvmlite 0.50.0 and ezdxf 1.4.4. Native Tk checks exercised all four methods, control activation, cancellation and close-with-worker cleanup without callback errors. The present layout was checked at 1400 by 662 pixels.

A recorded synthetic run with 1,000,000 observations, outlier removal enabled, Min Dist 2, Grid Spacing 10, local RBF and protection enabled produced 62,500 retained observations and 2,500 clipped grid nodes in approximately 7.30 seconds with compiled kernels cached. This is a historical environment-specific workflow measurement, not a performance guarantee or an independent accuracy assessment. The default outlier setting remains disabled.

26.3 Reproducible engineering runs

Keep the exact input, script, environment lock, settings and report together. Record the CRS, vertical datum, source-survey quality and any preprocessing outside this program. Repeat candidate settings on the same input when comparing methods. Inspect maps and profiles together with diagnostic flags, rather than ranking surfaces only by visual smoothness.

Fixed seeds and stable ordering improve repeatability, but exact bitwise identity across operating systems, library releases and BLAS/Qhull implementations is not promised. Export rounding may hide small internal differences. Automated tests establish behaviour on their cases; they do not certify a particular engineering dataset.

27. Method and parameter selection

27.1 General reconstruction

Start with the default protected RBF when a smooth local representation is useful and the retained cloud is large. Examine the proportion and spatial distribution of fallback and correction flags. If broad corrections occur, inspect sampling geometry and compare against linear TIN rather than merely increasing smoothing.

27.2 Sparse surveys and contour bands

Use protected RBF or linear TIN as a baseline between sparse sections. Keep the guard enabled for smooth methods. A small grid spacing adds interpolated detail to the file but does not create new survey information. Large gaps can remain poorly constrained even when every predicted elevation is locally bounded.

27.3 Smooth, well-supported data

Clough-Tocher can provide smooth interior derivatives on moderate control sets. Check whether its protection is active in areas where derivative smoothness is important. Turning protection

off restores the unmodified method more closely, but also permits its inherent overshoot.

27.4 Very dense data

Voxel mode can reduce redundant controls before triangulation. Choose a width consistent with the smallest feature that must be retained. Gaussian smoothing adds another length scale, approximately sigma times grid spacing, distinct from the aggregation width. Compare sigma zero and positive sigma when assessing whether a feature has been excessively smoothed.

27.5 Minimum distance, boundary and validation

Select Min Dist from spatial redundancy and feature size, not merely the desired output spacing. Inspect the footprint when datasets contain disconnected groups or interior exclusion zones. Keep independent holdouts enabled when there are enough controls, but supplement random holdouts with project-specific external checks for survey gaps or extrapolation.

Enable outlier removal only when local planar residual screening is appropriate for the data. A real rock, quay edge or abrupt seabed step can resemble an outlier under that model. Screening is not a replacement for survey quality control.

28. Limitations and interpretation

The program reconstructs one scalar field over Cartesian XY. It has no breakline enforcement, coastline recognition, barrier-aware distances, anisotropic physical covariance, kriging variance, measurement-error weighting, constrained optimization, contour preservation or automatic multi-surface separation. The guard detects geometric risk using fixed numerical thresholds rather than an estimated physical process.

The footprint may include unsurveyed regions. Local ranges suppress artificial extrema but do not recover unknown features, ensure positive water depth relative to a changing tide, guarantee monotonic profiles, conserve dredging volume or impose slope constraints. Smooth-method correction flags are algorithmic diagnostics, not automatic rejection criteria.

Random holdout errors are conditional on the conditioned data. The confidence index is uncalibrated. Source coordinate errors, vertical bias, instrument uncertainty, temporal seabed changes and geodetic datum errors are outside the numerical validation. Use the outputs with an understanding of their resolution and provenance.

The source is portable Python, while Tk appearance, available fonts and packaged executables are platform-dependent. The in-memory support structures can limit dataset size even though grid export is streamed. Normal cancellation and rollback are implemented, but abrupt process termination and filesystem failures can interrupt publication outside recoverable conditions.

29. Bibliography and primary documentation

The mathematical descriptions above explain the application and its selected library algorithms. The support-risk thresholds, blending policy, diagnostic formula and publication workflow are specific to this script. Library documentation describes backend behaviour; it should not be read as an independent validation of those application policies.

1. Fasshauer, G. E. (2007). *Meshfree Approximation Methods with MATLAB*. World Scientific. ISBN 978-981-270-634-8. Background on radial approximation and polynomial

augmentation. Publisher.

2. SciPy developers. *RBFInterpolator*. API reference, kernel definitions, smoothing, polynomial conditions and local-neighbour evaluation. Documentation.

3. SciPy developers. *CloughTocher2DInterpolator*. API reference, triangulated cubic construction and gradient estimation. Documentation.

4. Nielson, G. M. (1983). A method for interpolating scattered data based upon a minimum norm network. *Mathematics of Computation*, 40, 253-271. DOI.

5. Renka, R. J., and Cline, A. K. (1984). A triangle-based C^1 interpolation method. *Rocky Mountain Journal of Mathematics*, 14(1), 223-237. DOI.

6. SciPy developers. *LinearNDInterpolator*. API reference and linear barycentric interpolation. Documentation.

7. SciPy developers. *gaussian_filter*. Finite multidimensional Gaussian filtering, support radius and boundary modes. Documentation.

8. SciPy developers. *cKDTree*. Spatial-index construction and nearest-neighbour searches. Documentation.

9. SciPy developers. *binary_fill_holes*. Binary-domain hole filling. Documentation.

10. SciPy developers. *Delaunay* and *ConvexHull*. Qhull-backed planar support structures. Delaunay documentation; ConvexHull documentation.

11. ezdxf developers. *DXF Tags*. Group-code/value serialization and the ASCII DXF data model. Documentation.

12. Numba developers. *Compiling Python code with @jit*. JIT specialization and execution. Documentation.

13. Numba developers. *Just-in-Time compilation*. Options for native execution, GIL release, parallelism, caching and floating-point transformations. Documentation.

14. Python Software Foundation. *tkinter: Python interface to Tcl/Tk*. Documentation.

15. Python Software Foundation. *venv: Creation of virtual environments*. Documentation.

16. PyInstaller development team. *Using PyInstaller*. Bundle generation, console/windowed modes and collection options. Documentation.

17. threadpoolctl developers. *threadpoolctl*. Native library thread-pool control. Project documentation.

18. NumPy developers. *NumPy reference*. Arrays, ordering, aggregation and floating-point operations. Documentation.